

Assignment-19.1

Name:M.Saideep
HT.NO:2303A51763

Batch: 12

Task:

Translate a simple Python class into Java using AI assistance.

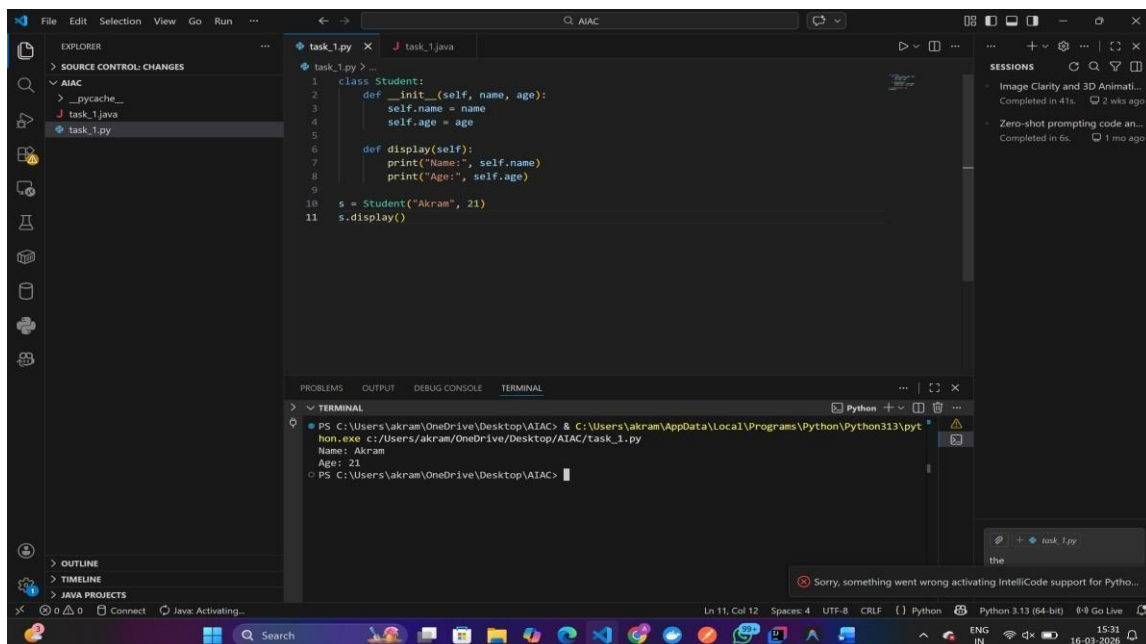
Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

CODE:

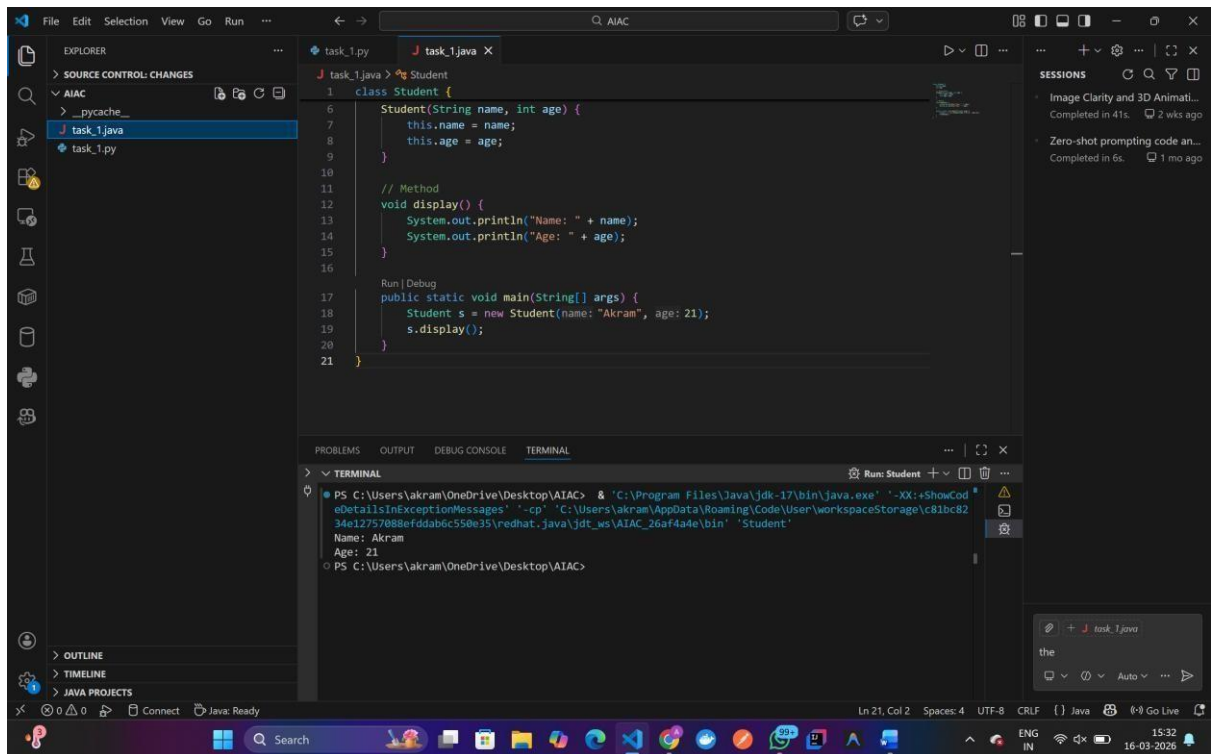


The screenshot shows a code editor with a Python file named `task_1.py`. The code defines a `Student` class with an `__init__` method that takes `name` and `age` as arguments, and a `display` method that prints the name and age. An instance `s` of the `Student` class is created with the name "Akram" and age 21, and the `display` method is called. The terminal output shows the execution of the script, which prints "Name: Akram" and "Age: 21".

```
1 class Student:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def display(self):
7         print("Name:", self.name)
8         print("Age:", self.age)
9
10 s = Student("Akram", 21)
11 s.display()
```

Terminal Output:

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```



Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

CODE:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)  
print(factorial(5))
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py  
120
```

```
#include <iostream>  
using namespace std;  
  
int factorial(int n) {  
    if(n == 0) {  
        return 1;  
    }  
    return n * factorial(n-1);  
}  
  
int main() {  
    cout << factorial(5);  
    return 0;  
}
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py  
120
```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

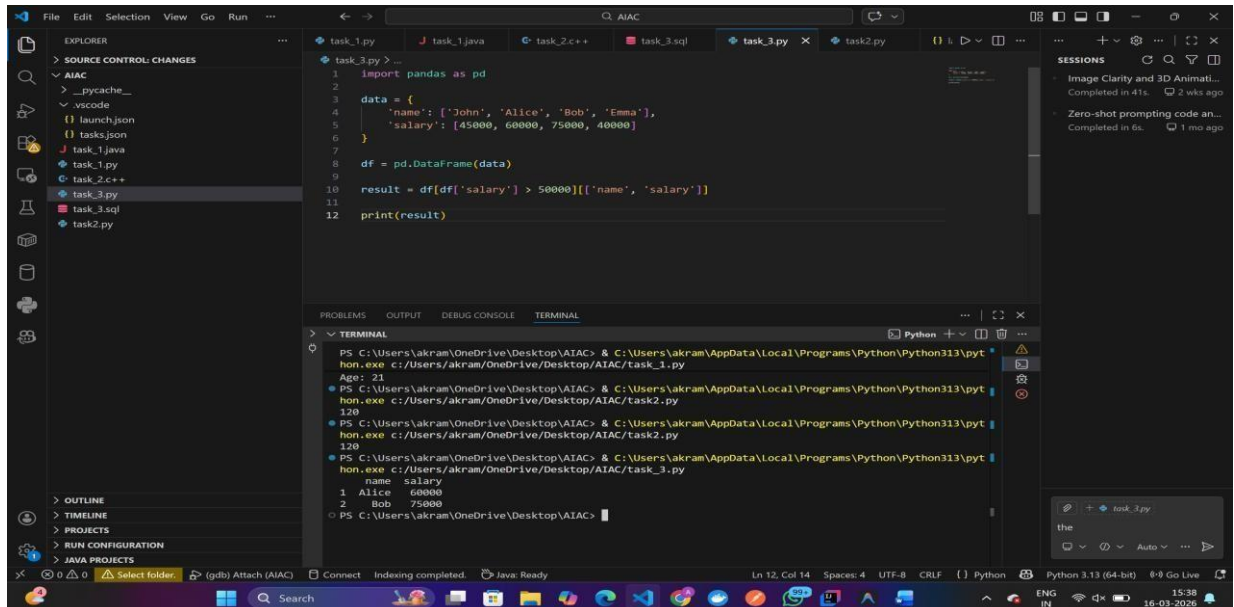
- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.

- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

CODE:



```

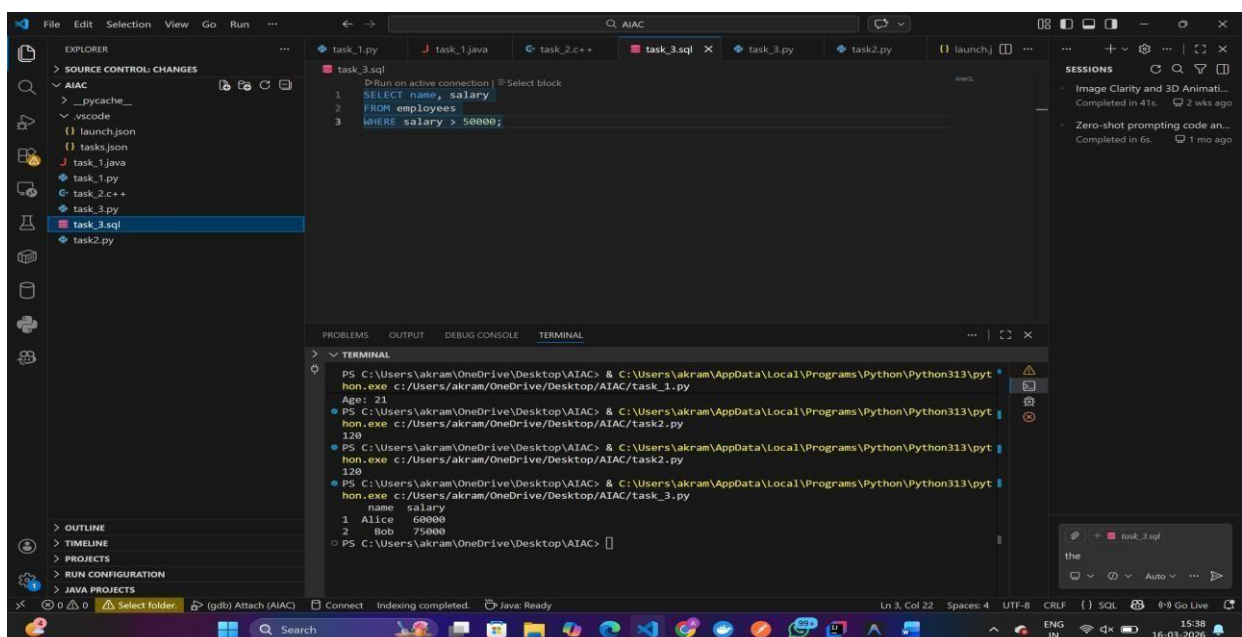
1 import pandas as pd
2
3 data = {
4     'name': ['John', 'Alice', 'Bob', 'Emma'],
5     'salary': [45000, 60000, 75000, 40000]
6 }
7
8 df = pd.DataFrame(data)
9
10 result = df[df['salary'] > 50000][['name', 'salary']]
11
12 print(result)

```

```

PS C:\Users\akram\OneDrive\Desktop\AIAC> C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_3.py
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_3.py
   name  salary
2  Bob    75000

```



```

1 SELECT name, salary
2 FROM employees
3 WHERE salary > 50000;

```

```

PS C:\Users\akram\OneDrive\Desktop\AIAC> C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_3.py
   name  salary
2  Bob    75000

```

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

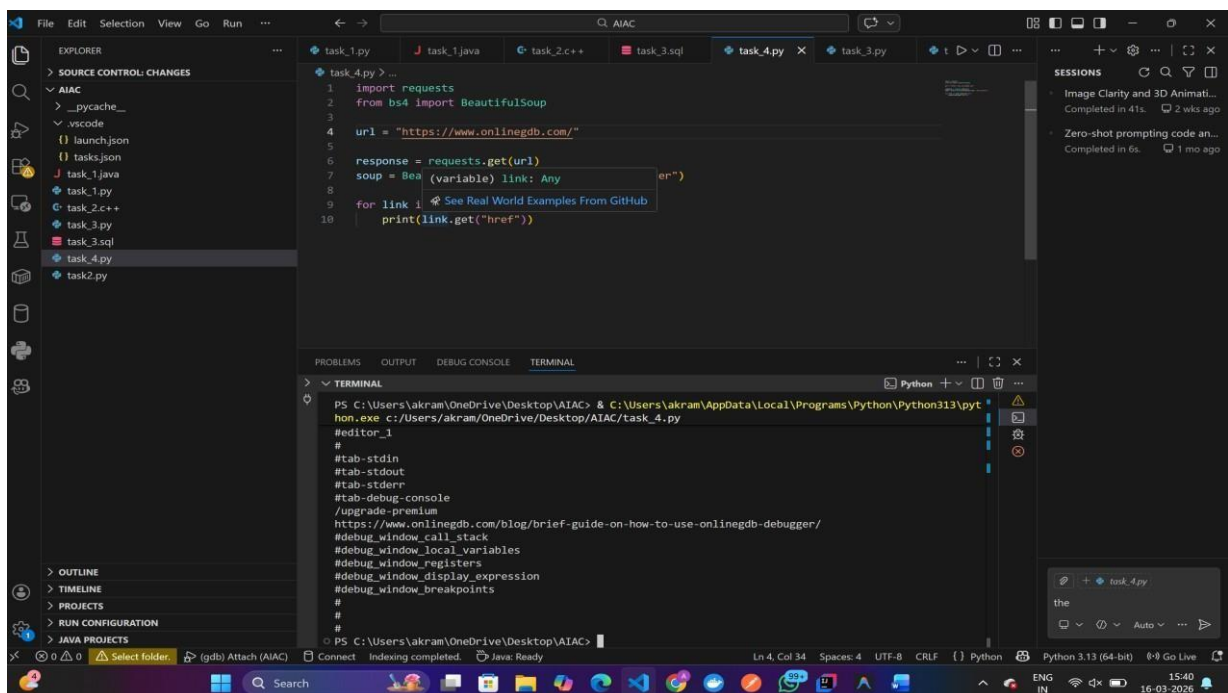
- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

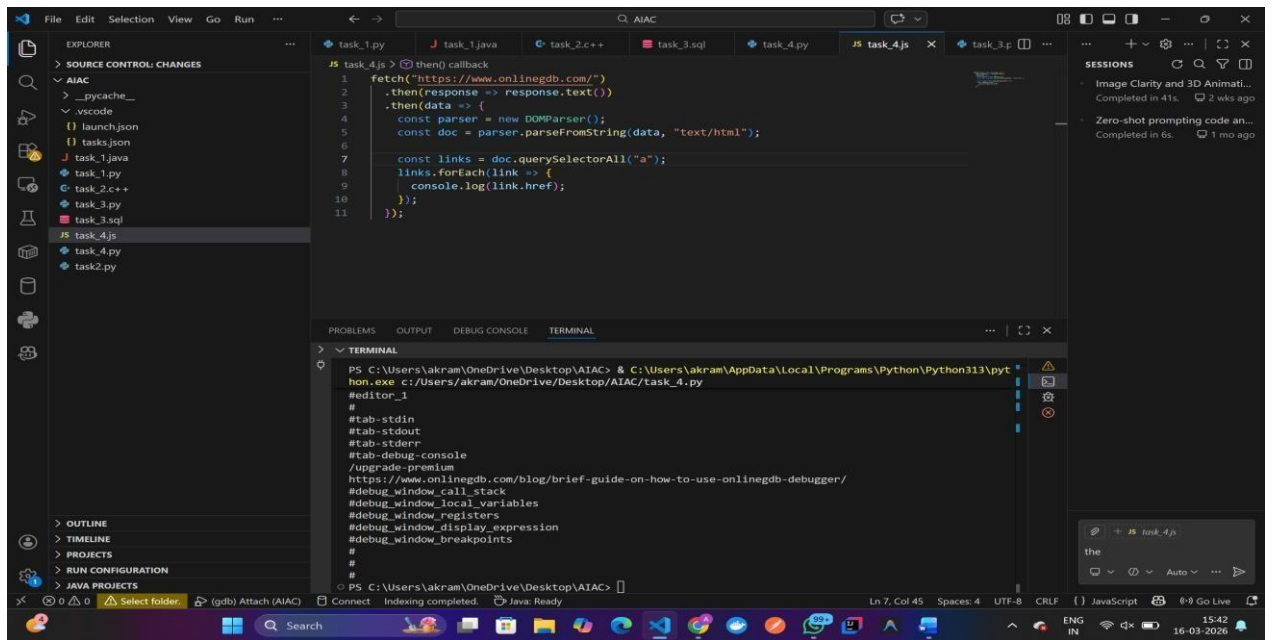
- Working equivalent code snippets in at least two different programming languages
- CODE:



The screenshot shows a Visual Studio Code editor with a file explorer on the left containing a project named 'AIAC'. The main editor window displays a Python script named 'task_4.py' with the following code:

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 url = "https://www.onlinegdb.com/"
5
6 response = requests.get(url)
7 soup = BeautifulSoup(response.text, 'html.parser')
8
9 for link in soup.find_all('a'):
10     print(link.get('href'))
```

The terminal window at the bottom shows the command prompt running the script, displaying the output of the web scraping process, including the URL of the page and various debug messages from the onlinegdb debugger.



Explanation: The experiment demonstrates that AI can effectively translate code between multiple programming languages such as Python, Java, C++, SQL, and Pandas, making development faster and more efficient. However, differences in syntax, data types, and libraries require careful verification and minor modifications after conversion. Python code is generally more concise compared to Java and C++, while Pandas provides a programmatic alternative to SQL queries. In real-time applications, challenges arise due to variations in frameworks and execution environments across languages. Overall, AI serves as a powerful tool for code conversion, but human understanding is essential to ensure correctness, optimize performance, and maintain logical consistency